

FEDERAL UNIVERSITY
OF SANTA CATARINA

Laboratório 4:
Atribuição Condicional

EEL5105 – Circuitos e Técnicas Digitais

Introdução

Tarefa

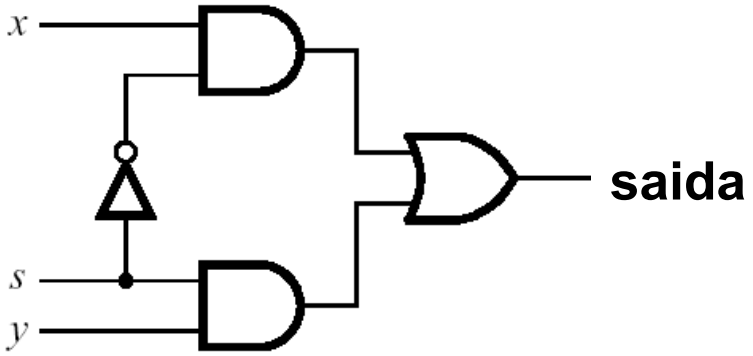
Emulação na placa DE1

Objetivos

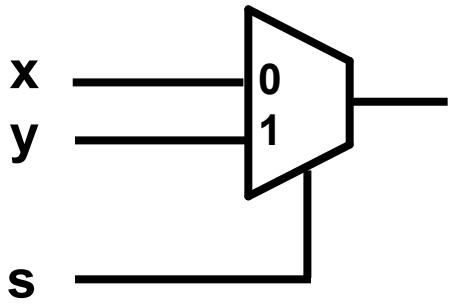
- Entender **atribuições condicionais** (usando estrutura **when/else** ou **with/select**) para facilitar o projeto de circuitos em **VHDL**.
- Trabalhar os conceitos de **multiplexador** e **decodificador**.
- Implementar **multiplexadores** e **decodificadores** em **VHDL** utilizando **atribuições condicionais**.
- Fazer um estudo de caso visando fixar os conceitos abordados.

Introdução

- **Exemplo 1: multiplexador com 2 entradas**



(a)



(b)

s	saida
0	x
1	y

(c)

VHDL convencional:

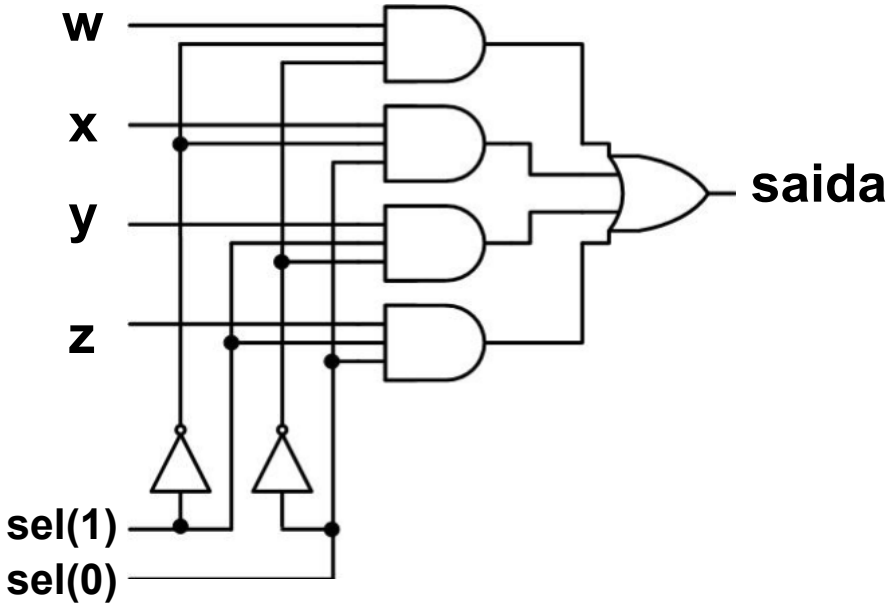
```
saida <= (x and (not(s)) or (y and s);
```

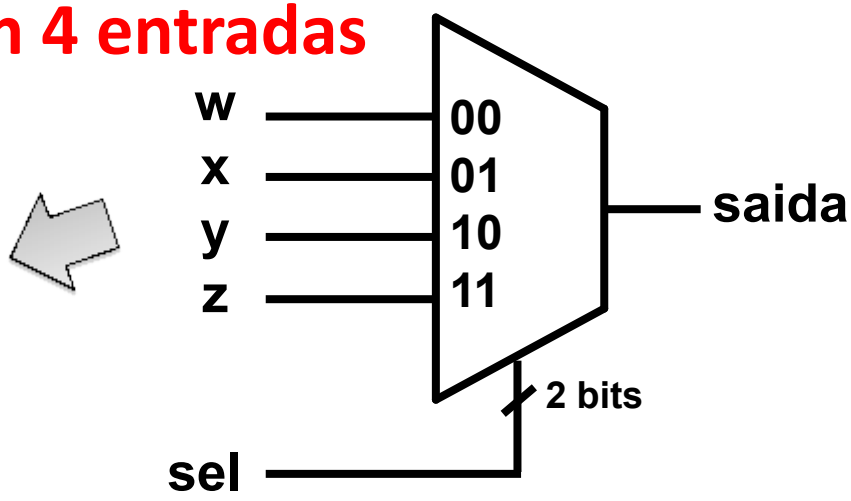
VHDL usando when/else:

```
saida <= x when s = '0' else
         y;
```

Introdução

- Exemplo 2: multiplexador com 4 entradas





Com when/else:

```
saida <= w when sel = "00" else
         x when sel = "01" else
         y when sel = "10" else
         z;
```

VHDL convencional:

```
saida <= (w and (not sel(1)) and (not sel(0))) or
         (x and (not sel(1)) and sel(0)) or
         (y and sel(1) and (not sel(0))) or
         (z and sel(1) and sel(0));
```

Introdução

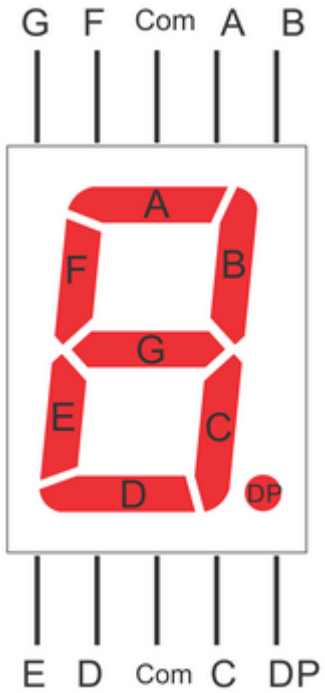
- **When/else** -- **With/select**: estruturas da linguagem **VHDL** que permite a **atribuição condicional** de valores a um **sinal**.
  - Facilita o projeto de **sistemas digitais**, pois permite a descrição de um **circuito** de forma análoga à sua **tabela verdade** sem a necessidade de **descrição booleana**.
  - **Não** implica execução **sequencial** de código (ou seja, não se trata de **if/else**).
  - Facilita a **interpretação** do código.
  - **Exemplo**:

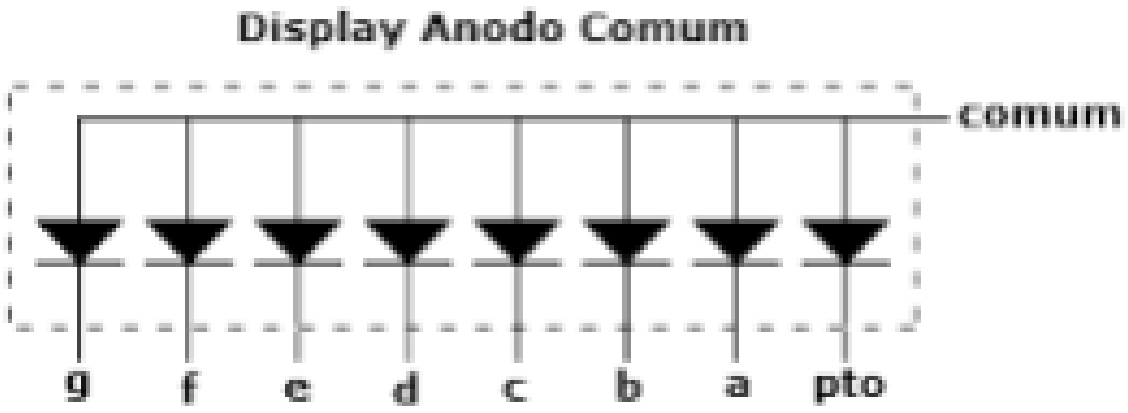
```
saida <= "0001" when entrada = "00" else
         "0010" when entrada = "01" else
         "0100" when entrada = "10" else
         "1000";
```

Introdução

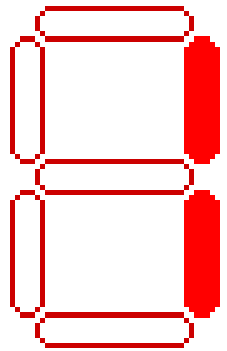
- O kit **DE1-SoC** possui 8 **displays** de 7-segmentos, todos do tipo ânodo comum (**LEDs acendem com zero lógico**).

Display de 7 segmentos - configuração ânodo comum





1	1	1	1	0	0	1	1

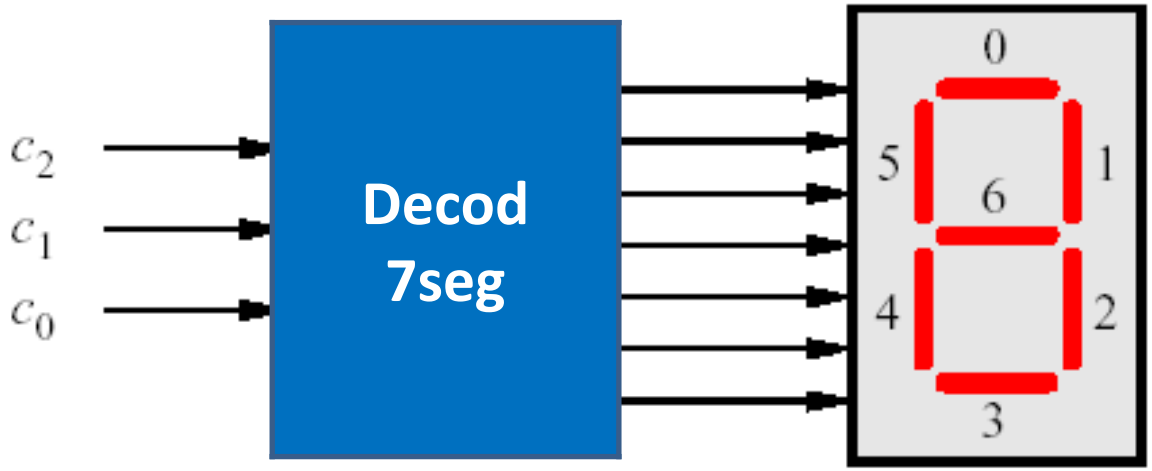


Introdução

- **Decodificadores**

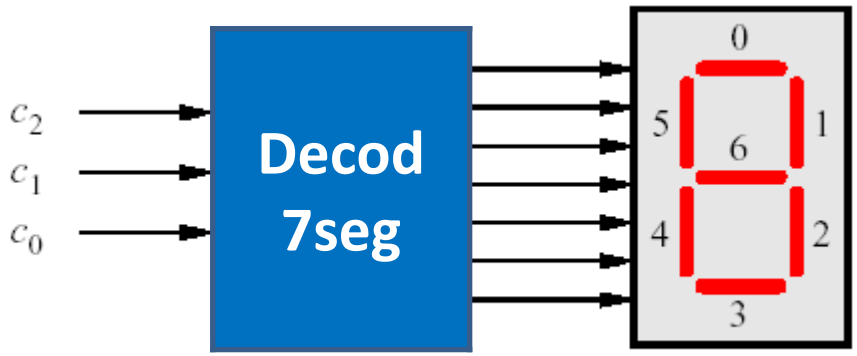
  - Para mostrar valor em um dos displays, é preciso realizar uma **conversão** do código binário para o código 7-segmentos.

  - Os 3 bits de entrada do circuito **"Decod7seg"** são *decodificados*, e a palavra de 7 bits gerada é enviada para o display de 7 segmentos.



Introdução

- **Decodificadores**
  - Exemplos de valores em binários convertidos (decodificados) para 7-segmentos, visando escrita no display da placa DE1 anodo comum).



C2	C1	C0	7 bits *6543210*	Letra
0	0	0	1000001	U
0	0	1	0001110	F
0	1	0	0010010	S
0	1	1	1000110	C
1	X	X	1111111	

- Nesse exemplo, ao receber "**000**" na entrada, o decodificador gera o código equivalente a presentação da letra "**U**" no display 7seg.
- Ao receber "**1XX**", todos os segmentos são desligados.

Introdução

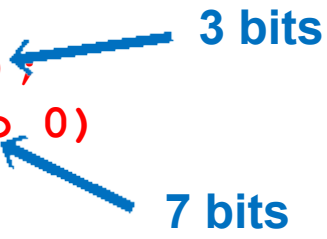
- **Decodificador UFSC (when/else):**

```
library IEEE;
use IEEE.Std_Logic_1164.all;

entity decodUFSC is
port (SW:   in std_logic_vector(2 downto 0);
      HEX0:  out std_logic_vector(6 downto 0)
     );
end decodUFSC;

architecture decod of decodUFSC is

begin
HEX0 <= "1000001" when SW = "000" else     -- U
        "0001110" when SW = "001" else     -- F
        "0010010" when SW = "010" else     -- S
        "1000110" when SW = "011" else     -- C
        "1111111";
end decod;
```



Introdução

- Decodificador UFSC (with/select):

```
library IEEE;
use IEEE.Std_Logic_1164.all;

entity decodUFSC is
port (SW:   in std_logic_vector(2 downto 0);
      HEX0:  out std_logic_vector(6 downto 0)
     );
end decodUFSC;

architecture decod of decodUFSC is

begin
 with SW select
   HEX0 <=          "1000001" when "000",        -- U
                    "0001110" when "001",        -- F
                    "0010010" when "010",        -- S
                    "1000110" when "011",        -- C
                    "1111111"  when others;
end decod;
```

3 bits

7 bits

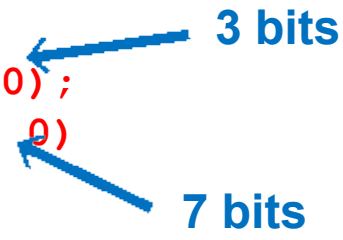
Introdução

- **Decodificador** (mostra valores de 0 a 7)

```
library IEEE;
use IEEE.Std_Logic_1164.all;

entity decoder is
port (SW:    in std_logic_vector(2 downto 0);
      HEX0:  out std_logic_vector(6 downto 0)
     );
end decoder;

architecture decod of decoder is
begin
  HEX0 <=      "1000000" when SW = "000" else    -- 0
               "1111001" when SW = "001" else    -- 1
               "0100100" when SW = "010" else    -- 2
               "0110000" when SW = "011" else    -- 3
               "0011001" when SW = "011" else    -- 4
               "0010010" when SW = "101" else    -- 5
               "0000010" when SW = "110" else    -- 6
               "1111000";                        -- 7
end decod;
```



Introdução

Tarefa

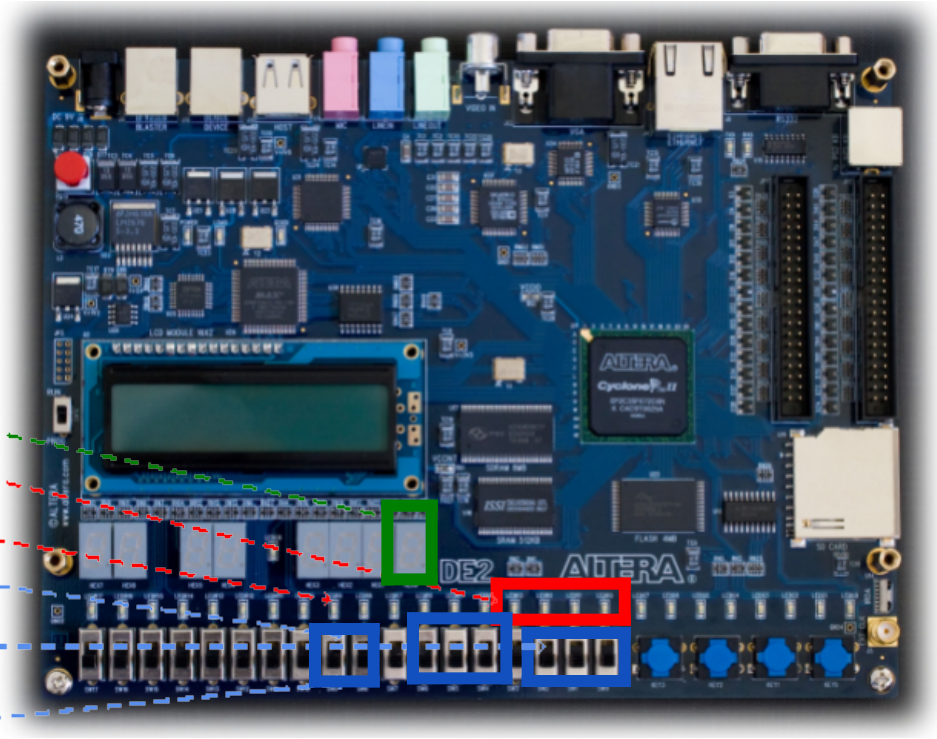
Emulação na placa DE1

Tarefa

- Implementar um componente que realiza 4 operações aritméticas sem sinal com entradas de três bits e saída de 4 bits. As operações são selecionadas mediante um sinal de controle de 2 bits.

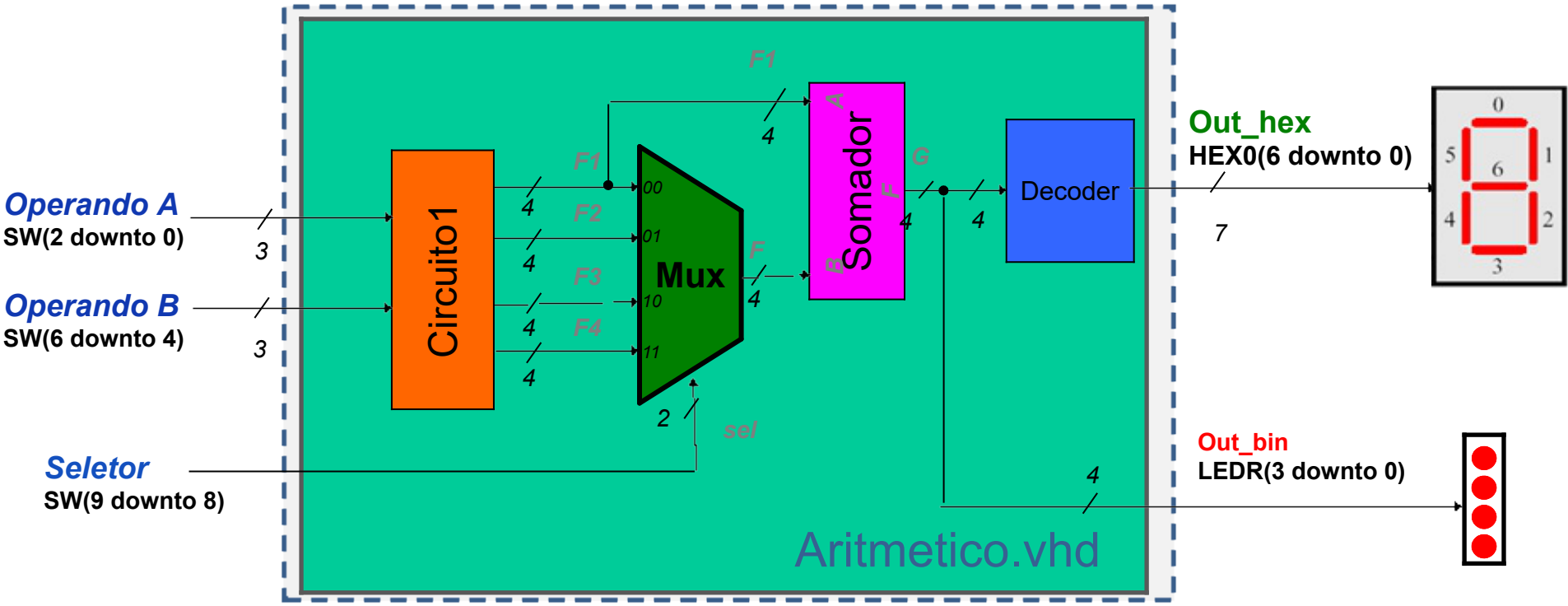
Tabela de operações

Seletor	Saída
00	2A
01	3A
10	A+B
11	A+2B



Tarefa

- O circuito vai ser implementado usando um **Multiplexador**, um **Somador** e um **Decoder** como mostrado abaixo.



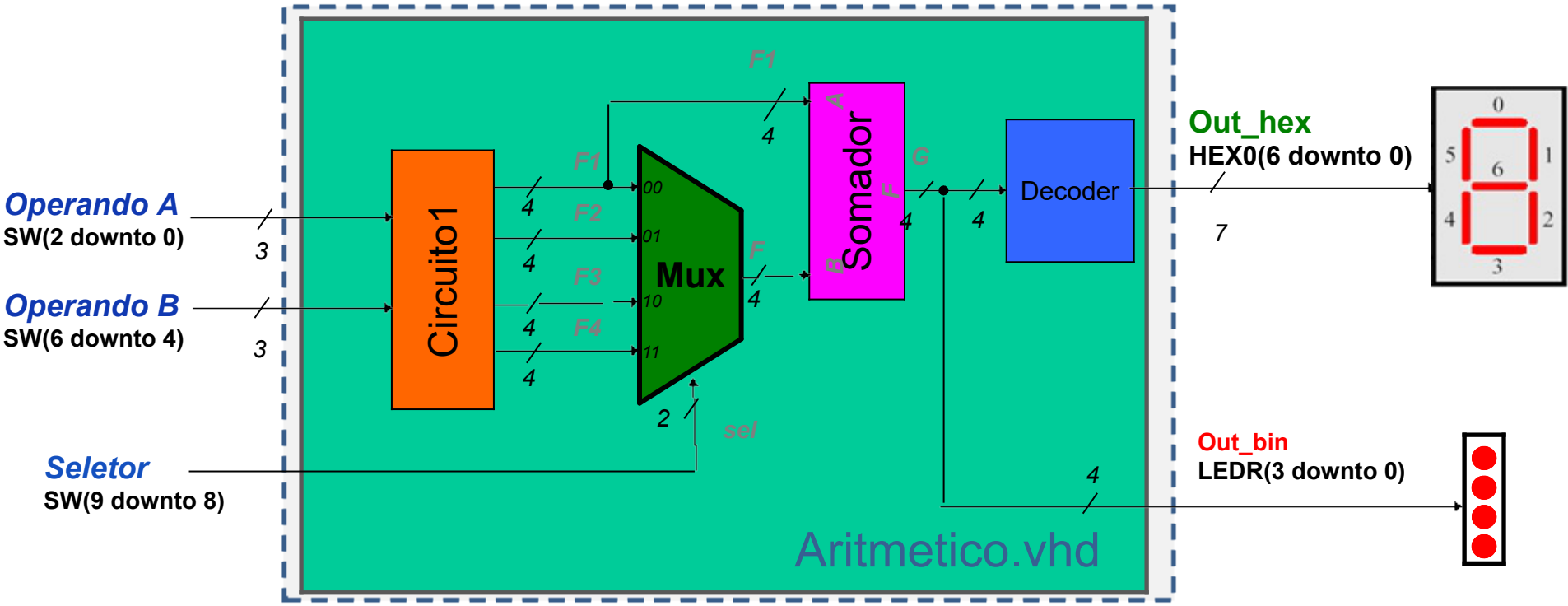
Tarefa

- **Passo 1**: Criar projeto no **Quartus II**
  - Acessar **File -> New Project Wizard** e criar um projeto com as seguintes características:
    - **Pasta**: nomear a pasta como ***AULA4/***
    - **Sugestão de nome do Projeto**: **Aritmetico**
    - **Tipo**: *Empty Project*
    - Em ***Add Files***, não adicionar nenhum arquivo
    - Demais opções deixar como padrão

- **Passo 2**: Criar arquivos do tipo **VHDL** dentro do projeto
  - No **Quartus II**, acessar
    File -> New -> Design Files -> VHDL File
    e o arquivo "***Vhdl1.vhd***" será criado:

Tarefa

- **Passo 2.1:** Implementar em VHDL o bloco **Somador** que realiza a soma das entradas de 4 bits (próximo *slide*).



Tarefa

```
library IEEE;
use IEEE.Std_Logic_1164.all;
use IEEE.std_logic_unsigned.all; -- necessário para o operador de soma: +

entity Somador is
port (
        A: in std_logic_vector(3 downto 0);
        B: in std_logic_vector(3 downto 0);
        F: out std_logic_vector(3 downto 0)
    );
end Somador;

architecture Som_arch of Somador is
begin

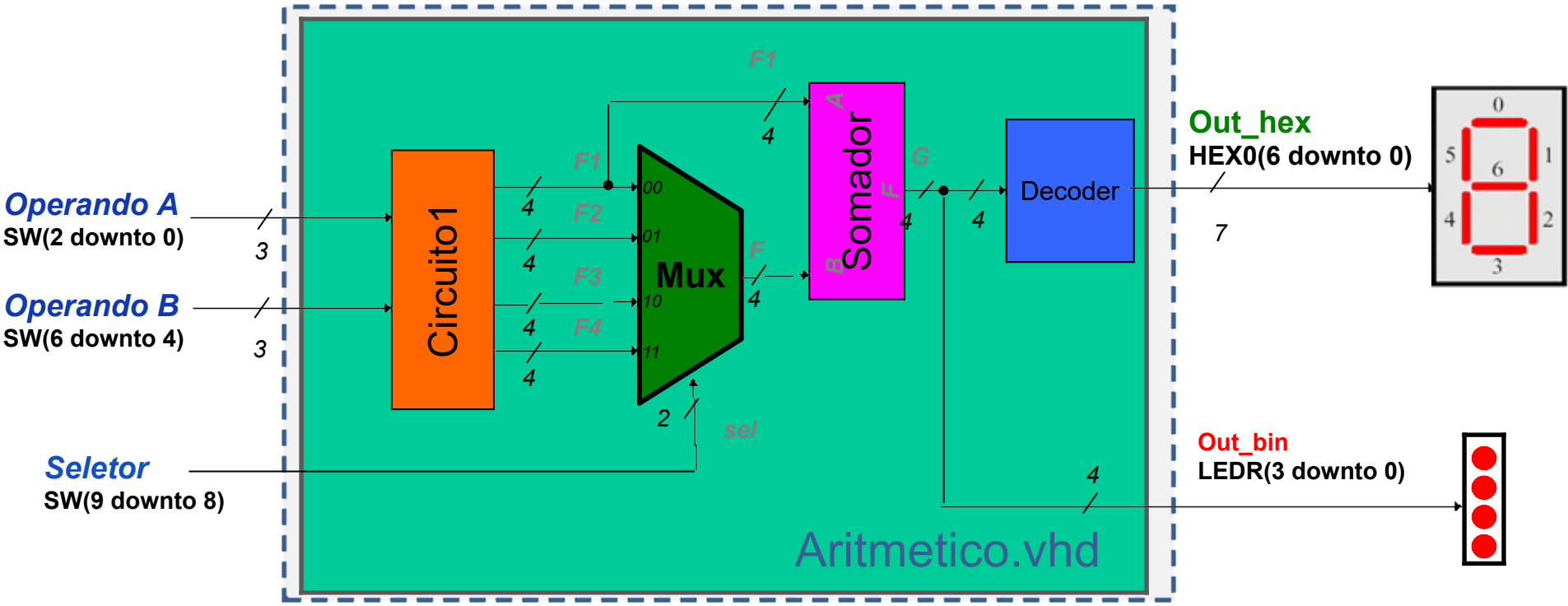
        F <= A + B;

end Som_arch;
```

Tarefa

- **Passo 2.2:** Implementar, em VHDL, o bloco **Multiplexador** usando as orientações do **slide 5**

- **Passo 2.3:** Implementar em VHDL o bloco **Decoder**. Para visualizar os resultados em hexadecimal, um **Decoder** deverá ser projetado usando (e completando) a tabela a seguir e usando exemplo do **slide 12**.



Tarefa

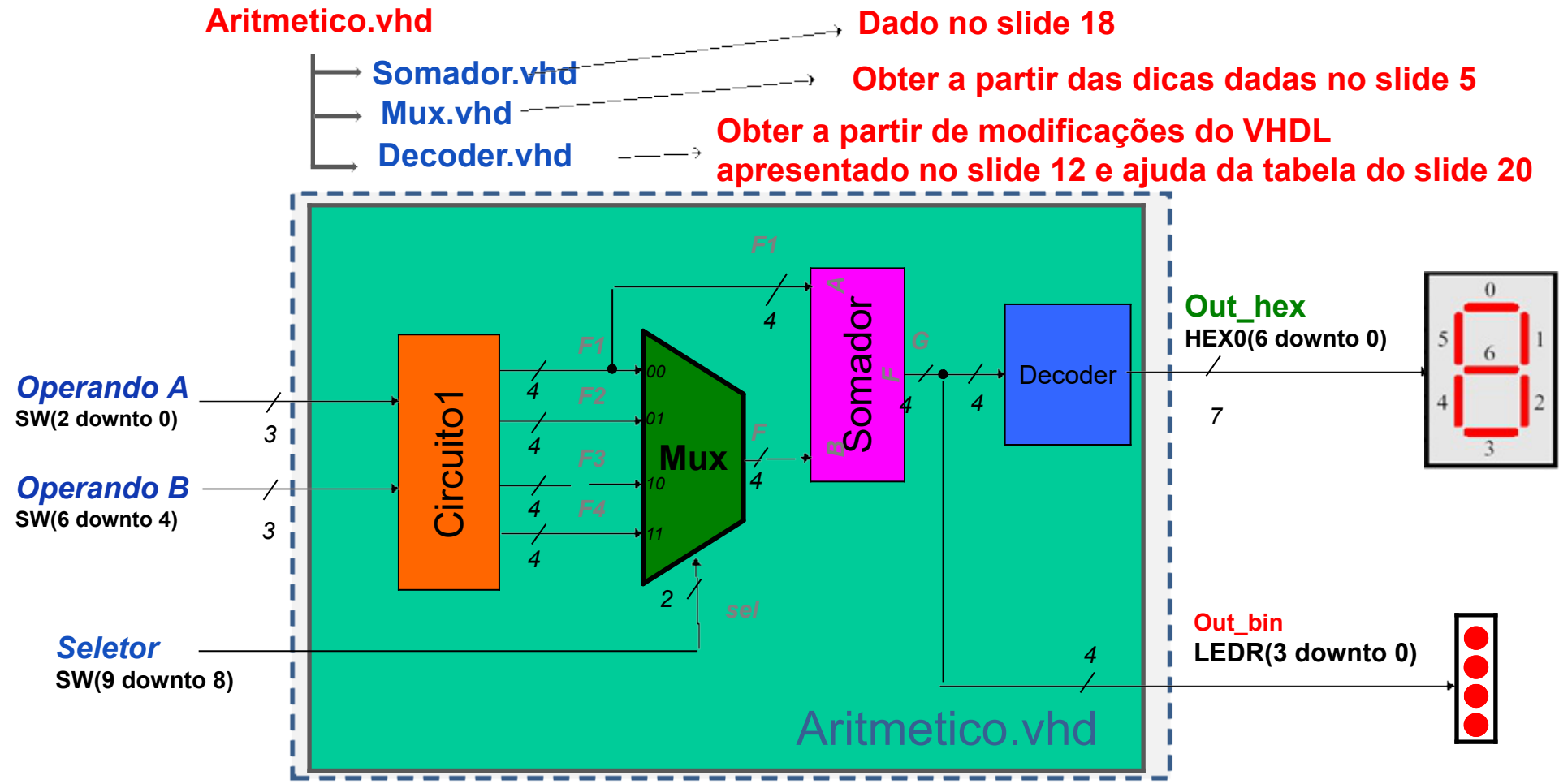
- A entrada do **Decoder** terá 4 bits; a saída, 7 bits. Aqui é mostrada a tabela com 11 das 16 combinações possíveis. Cabe ao aluno preencher os elementos que faltam na tabela e implementar em VHDL.

Entrada	Saída 6543210	Display
0000	1000000	0
0001	1111001	1
0010	0100100	2
0011	0110000	3
0100	0011001	4
0101	0010010	5
0110	0000010	6
0111	1111000	7
1000	0000000	8
...	...	9, A, b, C, d
1110	0000110	E
1111	0001110	F

Fazer

Tarefa

- **Passo 2.4:** Criar o circuito **Aritmetico.vhd** aplicando os conceitos aprendidos na aula do laboratório 3 sobre projeto hierárquico.



Tarefa

A *entity* deve ser a seguinte:

```
entity Aritmetico is

        port (
                SW: in std_logic_vector(9 downto 0);
                 LEDR: out std_logic_vector(3 downto 0);
                 HEX0: out std_logic_vector(6 downto 0)
               );

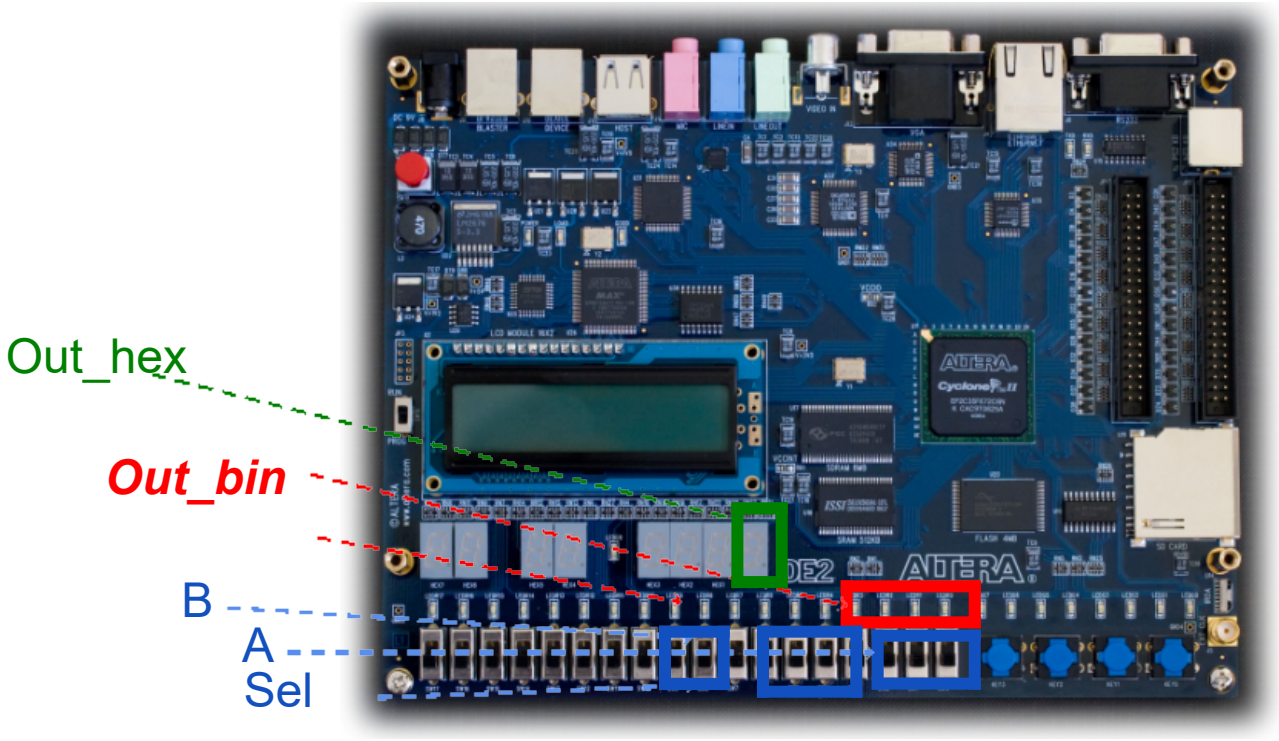
end Aritmetico;

architecture Arit_arch of Aritmetico is


...
end  Arit_arch;
```

Tarefa

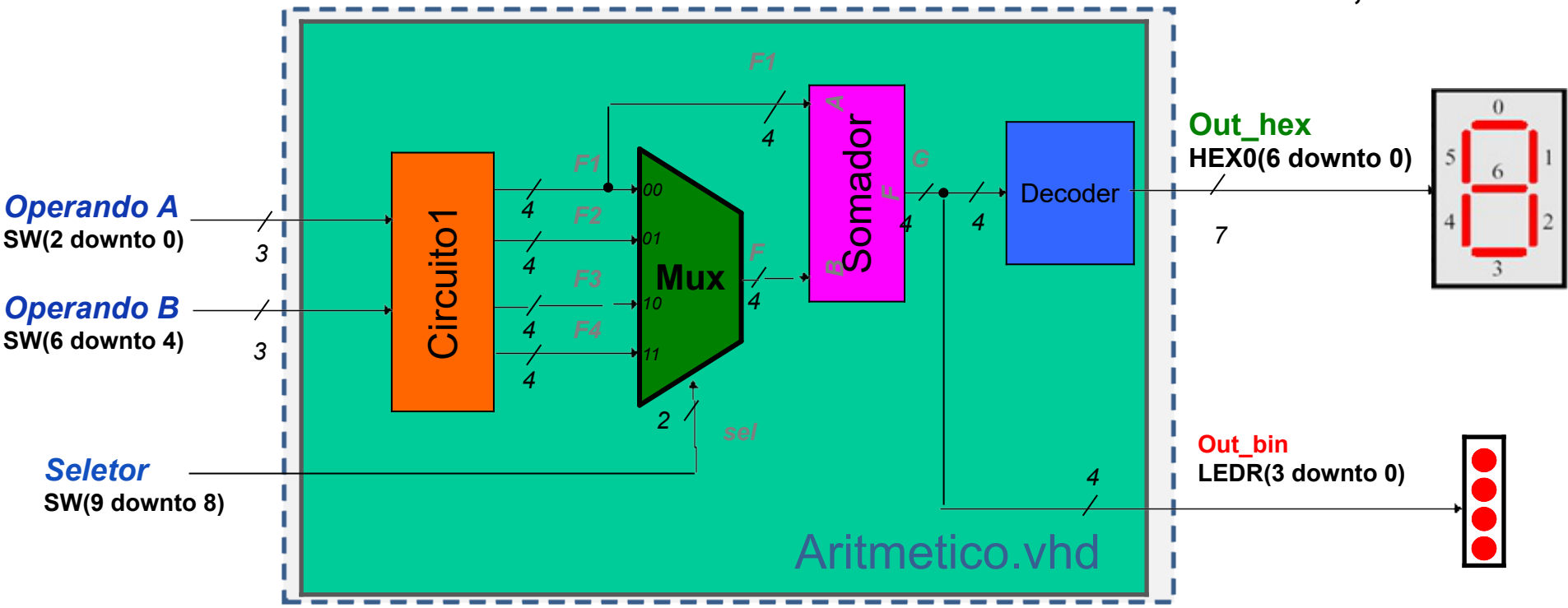
Esta declaração de entradas e saídas na entity do VHDL permite a emulação. Assim, as entradas **A** e **B** devem ser associadas a **SW(2 downto 0)** e **SW(6 downto 4)**, **Control** a **SW(9 downto 8),** e a apresentação dos resultados **out_bin** na saída **LEDR(3 downto 0)** e **out_hex** display **HEX0**.



Tarefa

- **Passo 2.5:** Implementar em VHDL o bloco denominado **Circuito1** de acordo as seguintes linhas de código:

```
F1 <= '0' & A;  --A
F2 <= A & '0';  --2A
F3 <= '0' & B;  --B
F4 <= B & '0';  --2B
```



Tarefa

- **Passo 3**: **Compile** o projeto **Aritmetico** e **analise os erros** ocorridos.

- **Passo 4**: Uma vez compilado sem ocorrência de erros de sintaxe, **simular** o circuito no ***ModelSim*** da seguinte forma.

  - A=3 e B=4 para as 4 operações possíveis (2A, 3A, A+B e A+2B).
  - A=2 e B=7 para as 4 operações possíveis (2A, 3A, A+B e A+2B).

- **Passo 5:** Visualize o diagrama de blocos gerado **Tools-> Netlist Viewers-> RTL Viewer.**

Introdução
Tarefa

Emulação na placa DE1

Emulação na placa DE1

6.1 (Emulador on-line): Abra o emulador https://fpgaemu.sites.ufsc.br/login e faça ***UPLOAD*** dos arquivos VHDL que deseja utilizar na emulação. Defina *aritmetico.vhd* como topo da hierarquia usando o botão ***SET TOP LEVEL*** e compile. Em seguida, selecione o emulador, inicie a emulação e verifique o funcionamento.

6.2 Faça as 4 operações possíveis (2A, 3A, A+B e A+2B). **Entenda os resultados obtidos no Lab4.doc**.